

How to Approximate A Set Without Knowing Its Size In Advance

241880161 仇浩嘉 241880136 任春驿 241880070 周文成

2026 年 7 月 29 日

目录

1 概述	2
2 引言	2
3 预备知识	5
4 空间下界	5
5 构造一：几何增长数据结构	9
6 构造二：常数时间查询	11
7 相关研究	13
8 总结与讨论	15

1 概述

本文 *How to Approximate A Set Without Knowing Its Size In Advance* (Pagh, Segev, Wieder, 2013) 研究的是动态近似成员查询问题的一个基本变体：当待表示集合的最终大小 n 事先未知时，数据结构必须支持在线插入和成员查询（无假阴性、假阳性率 $\leq \varepsilon$ ），并且空间尽可能接近信息论下界。

论文的核心贡献由三部分组成。其一，通过压缩论证证明了空间下界 $(1 - o(1))n \log(1/\varepsilon) + \Omega(n \log \log n)$ ，首次揭示了“已知 n ”与“未知 n ”两种设定之间的超线性空间差距。其二，提出构造一（几何增长数据结构），以 $O(\log n)$ 查询时间为代价，达到 $(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$ 的空间上界。其三，提出构造二（单字典前缀扩展），将查询时间改进为最坏情况 $O(1)$ ，同时保持相同量级的空间上界；并通过去摊销化进一步给出插入最坏情况 $O(1)$ 的版本。

本 review 按论文原有章节顺序展开：引言与预备知识 (§1-2)、空间下界 (§3)、上界构造一和构造二 (§4-5)、相关研究 (§6)，最后以总结与讨论收束 (§7)。

2 引言

2.1 问题定义

近似成员数据结构：给定一个可以在线插入元素的集合 S （大小为 n ），支持成员查询，并且需要满足以下这两个条件：

- 无假阴性： $\forall x \in S$ ，查询总是返回 yes。
- 假阳性率最多 ε ： $\forall x \notin S$ ，查询返回 yes 的概率最多达到给定的 ε 。

当 n 已知的时候，已被证明空间占用是 $\Theta(n \log(1/\varepsilon))$ bits，单个元素占用 $\Theta(\log(1/\varepsilon))$ bits。

已有的近似成员数据结构需要集合 S 的大小 n 已知，该论文研究的是 n 未知的情形。最终集合大小 n 不定，需要考虑结构自适应扩容，且要维持空间尽可能小。传统最优空间理论在这个设定下不再直接适用。

2.2 问题为什么重要

本文及所引用的 [BM03] 综述指出，近似成员查询在大量场景中广泛使用，这些场景中集合大小 n 通常无法提前准确预知，亟需一个解决方案。

1. 网络流量监控与包路由。路由器按源/目的 IP + 端口进行计数，一个时间段内不同流的数量取决于用户行为、攻击流量、突发事件，不可预测。若 Bloom Filter 容量固定，低负载时浪费 SRAM，高负载时假阳性率过大导致错误路由决策。
2. LSM-tree 键值存储。LevelDB、RocksDB、Cassandra 等系统在每层 SSTable 上使用 Bloom Filter 避免无效磁盘查找。写入量由应用负载决定，各层大小随 compaction 策略动态变化。提前分配固定容量意味着要么浪费内存，要么在数据增长后重建整个过滤器。

3. 分布式集合调和与数据同步。节点间交换键集差异时用 Bloom Filter 压缩传输，节点间数据分布不均匀，某时刻持有的键数量无法提前知道。固定容量迫使发送方低效地估算容量或多次往返。
4. 入侵检测与病毒扫描。需要将已知攻击签名存入 Bloom Filter 以在线匹配入站包。攻击签名库持续增长（新漏洞、变种），增长速率不可预测，需动态追加新签名而不重建全库。
5. 网页缓存与 CDN。缓存代理用 Bloom Filter 记录已缓存 URL 以快速判断命中。URL 集合大小随用户请求动态变化，新 URL 会持续出现，且缓存过期要求删除支持。

这些场景的共同特征是：数据以流式到达、 n 未知、内存有限、假阳性率需保持可控。论文指出，已有动态方案（如 Scalable Bloom Filter）在实际中已被用于此类场景，但其空间消耗高达 $\Omega(n \log n)$ 。本文首次从理论上证明了 $\Omega(n \log \log n)$ 的额外开销是不可避免的，并给出了达到该下界的紧致构造，为后续实用系统的设计提供了理论基础。

2.3 论文的主要贡献

论文在集合大小 n 事先未知的设定下，给出了近似成员查询问题的空间下界和上界，并给出两种有价值的构造。

2.3.1 定理 1.1（下界）

任意针对未知大小集合 S 、假阳性率 ε 的近似成员数据结构，在某 $n > u^\delta$ 次插入后，必须使用至少

$$(1 - o(1))n \log(1/\varepsilon) + \Omega(n \log \log n)$$

位的空间。

- 第一项 $(1 - o(1))n \log(1/\varepsilon)$ 是已知 n 时就已经需要的。
- 第二项 $\Omega(n \log \log n)$ 是“不知道 n ”所付出的额外代价。

注：条件 $n > u^\delta$ 的意思是 $\delta \in (0, 1)$ 为任意小常数， n 与全集大小 u 之间呈多项式关系（如 $n > \sqrt{u}$ 或 $n > u^{0.01}$ ），排除 n 极小（如 $n = O(\log u)$ ）的退化情况。需要注意，相比传统已知规模的场景，这里每元素平均开销多出了 $\Omega(\log \log n)$ bits，意味着开销会随着集合规模增长，无法维持常数单元素存储。在实际场景中假阳性率常取一个不太小的常数（如 $\varepsilon = 1/10$ ），此时下界要求每元素 $\Omega(\log \log n)$ 比特，而不是传统设定中的常数。

2.3.2 定理 1.2（上界）

论文给出两种匹配下界的构造，以及构造 2 的去摊销优化版本：

属性	构造 1 (热身)	构造 2 (精妙)	构造 2 (去摊销)
空间	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$O(n \log(1/\varepsilon) + n \log \log n)$
假阳性率	$\leq \varepsilon$	$\leq \varepsilon + u^{-c}$	$\leq \varepsilon + u^{-c}$
假阴性	无	无	无
插入时间	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
查询时间	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
是否需要已知 n	否	否	否

2.3.3 与 Bloom Filter 的对比

	Bloom Filter	构造 1	构造 2	构造 2 去摊销
空间 (每元素)	$1.44 \log(1/\varepsilon)$	$(1 + o(1)) \log(1/\varepsilon) + O(\log \log n)$	同构造 1	$O(\log(1/\varepsilon) + \log \log n)$
已知 n ?	是	否	否	否
查询时间	$O(\log(1/\varepsilon))$	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
插入时间	$O(\log(1/\varepsilon))$	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
空间是否紧致	否 ($1.44\times$)	是	是	否 (常数损失)
结构数量	1	$\log n$	1	1
桶化	不适用	不需要	用 ($u^{\delta/2}$ 桶)	不能用
依赖的哈希	k 个独立哈希	通用哈希	成对独立	成对独立
支持删除	否	否	可 (扩展)	可 (扩展)

下界为 $\Omega(n \log(1/\varepsilon) + n \log \log n)$ ，上界为 $O(n \log(1/\varepsilon) + n \log \log n)$ ，两者渐近阶完全一致。说明未知规模带来的 $\log \log n$ 的平均开销是不可避免的，也因此不存在更低阶的数据结构。

2.4 技术的非凡之处

1. 揭示了已知 n 和未知 n 之间的超线性空间差距—— $\Omega(n \log \log n)$ 是之前未被证明的固有代价。
2. 下界和上界紧致匹配，仅差低阶项。
3. 构造二同时做到空间逼近下界、查询 $O(1)$ 最坏、插入摊还 $O(1)$ ，三个维度均达到最优。
4. 构造二中 g_i 的 $\log \log u$ 位标签能精确控制翻倍次数，避免了上述过大过小估计引发的问题。
5. 支持去摊销优化能将插入从期望摊还升级到最坏常数，为实时系统提供理论保证。

3 预备知识

3.1 Word RAM 模型与记号

全集大小为 u ，单个元素占一个字，字长 $w = \lceil \log u \rceil$ bits。元素加减、按位布尔运算、任意位数的移位、整数乘法都是单位耗时。这是数据结构上下界证明的通用标准模型。下界证明不关心操作时间，只在意存储空间比特数。

3.2 k 重独立哈希函数族

$H : U \rightarrow V$ 是 k -wise independent 的，如果任取 k 个互不相同的 $x_1, \dots, x_k$ 和任意 $y_1, \dots, y_k \in V$ ：

$$\Pr_{h \leftarrow H} [h(x_1) = y_1 \wedge \dots \wedge h(x_k) = y_k] = \frac{1}{|V|^k}$$

弱化版 k -wise δ -dependent: $(h(x_1), \dots, h(x_k))$ 与均匀分布的统计距离不超过 δ ，允许微小偏差以降低实现开销。

论文中构造 2 只需要 pairwise independent ($k = 2$)，计算常数时间，仅需 $O(\log u)$ 位随机种子。

3.3 概率工具

- **马尔可夫不等式**：非负 $X \geq 0$ ， $\Pr[X \geq a] \leq \mathbb{E}[X]/a$ 。引理 3.2 中用来筛选优质随机种子： $\mathbb{E}[\mu(\hat{S}_r)] \leq 2\varepsilon$ ，取 $a = 4\varepsilon$ ，得 $\Pr[\mu \geq 4\varepsilon] \leq 1/2$ 。
- **切尔诺夫界**：独立伯努利变量和偏离期望的概率指数衰减，精度远高于马尔可夫。引理 3.4 中， $k = |C_i \cap \hat{S}_{i-1}|$ 期望 $\leq 4\varepsilon|C_i|$ ，取 $(1 + \delta)\mu = 9\varepsilon|C_i|$ 得尾概率 $\exp(-|C_i|)$ ， $|C_i| = n^{\Omega(1)}$ 时趋近于 0。
- **联合界**： $\Pr[\bigcup_i A_i] \leq \sum_i \Pr[A_i]$ 。构造 1 中用于总假阳性控制 ($\sum \varepsilon_i \leq \varepsilon$)。
- **无损编码信息论下界**： M 条互不相同的序列用无损前缀编码，至少需要 $\log_2 M$ bits。合法序列集合规模 $\geq u^n/3$ ，所以编码总长度必须 $\geq \log(u^n/3)$ 。用于压缩论证。

4 空间下界

4.1 直觉

对于精确字典，处理未知 n 很简单：每次插入一个新元素，字典描述从 S 更新到 $S \cup \{x\}$ 。近似成员结构不能这样做：它维护的是一个超集 $\hat{S}$ ，其中包含 S 本身和被误判为 Yes 的假阳性元素。插入新元素 x 后， $\hat{S}$ 必须变成更大的 $\hat{S}'$ ，且 $\hat{S} \subseteq \hat{S}'$ 。论文的核心发现是： $\hat{S}'$ 必须显著地比 $\hat{S}$ 大，需要加入许多元素。若 $\hat{S}' \setminus \hat{S}$ 太小，就可以利用数据结构的状态把整个序列 S' 压缩编码到信息论下界以下，推出矛盾。

4.2 思路

压缩论证：把数据结构当成编码工具，如果空间太小，就能用数据结构的状态把插入序列压缩到信息论下界以下，推出矛盾。证明的核心链路是：随机结构转确定性 → 指数分段 → 找测度增长最慢的一段 → 控制重叠元素 → 构造四段无损编码 → 化简得下界 → 参数赋值得主定理。

4.3 详细推导

我们定义几个东西：

- 随机比特串 r ：数据结构内部所有哈希、随机选择依赖的只读随机源，存储空间不计入结构开销。
- 确定性结构 B_r ：将随机源 r 固定后，整个插入、查询流程不再存在任何随机行为，输入完全决定输出。
- 判定超集 $\hat{S}_r$ ：对于插入序列 S ，所有被 B_r 查询返回“存在”的元素构成的集合。无假阴性保证 $S \subseteq \hat{S}_r$ 。
- 归一化测度 $\mu(\hat{S}_r) = |\hat{S}_r|/u$ ：判定超集在全集 U 中的占比，等价于随机抽一个元素被判 Yes 的概率。
- 合法序列集合 $S_{r,\varepsilon} = \{S \in U^n \mid \mu(\hat{S}_r) \leq 4\varepsilon\}$ ：固定随机种子 r 后，假阳性概率不超过 4ε 的那些插入序列。

4.3.1 随机结构转化为确定性结构（引理 3.2）

引理 3.2：存在随机串 r^* ，使 $|S_{r^*,\varepsilon}| \geq u^n/2$ 。

证明：

1. 对任意长度 n 的插入序列 S ，原始随机结构要求假阳性概率 $\leq \varepsilon$ ，所以：

$$\mathbb{E}_{r \leftarrow \{0,1\}^*} [\mu(\hat{S}_r)] \leq \varepsilon + \frac{n}{u}$$

文中限定 $n \leq \varepsilon u$ ，所以 $n/u \leq \varepsilon$ ，右式 $\leq 2\varepsilon$ 。

2. 用马尔可夫不等式： $\mu(\hat{S}_r) \geq 0$ 为非负随机变量，取 $a = 4\varepsilon$ ：

$$\Pr [\mu(\hat{S}_r) \geq 4\varepsilon] \leq \frac{\mathbb{E}[\mu(\hat{S}_r)]}{4\varepsilon} \leq \frac{2\varepsilon}{4\varepsilon} = \frac{1}{2}$$

3. 全空间 U^n 有 u^n 条插入序列。对任意 r ，落入 $S_{r,\varepsilon}$ 的序列比例至少 $1/2$ 。所以存在某个 r^* 使得至少一半的序列满足 $\mu(\hat{S}_{r^*}) \leq 4\varepsilon$ 。

4.3.2 指数分段与压缩论证 (引理 3.3、3.4)

拿到确定性结构 B_{r^*} 与合法序列集合 $\mathcal{S} = S_{r^*, \varepsilon}$ 后, 开始压缩论证。对任意长度为 n 的序列 S , 做指数分段:

- 分段基数 $\gamma \geq 2$ (自定义参数, 控制分段长度增长速度)。
- 第 i 段子序列 C_i 长度严格为 γ^i 。
- 前缀 S_i : 前 i 段拼接, 长度 $n_i = \sum_{j=1}^i \gamma^j$ 。
- 超集单调性: $\hat{S}_1 \subseteq \hat{S}_2 \subseteq \dots$, 每次插入新元素只能扩大判定超集, 所以测度单调不减: $0 \leq \mu(\hat{S}_1) \leq \dots \leq 4\varepsilon$ 。

分段的目的: 把完整的插入流程拆成指数增长的片段, 从中截取一段 C_i 作为编码论证的分析单元, 用总测度增量平摊到各段来得到单段增量的上界。

引理 3.3: 对任意 $S \in \mathcal{S}$, 存在 i 满足前缀长度 $n_i \in [\alpha n, n]$, 且:

$$\mu(\hat{S}_i) - \mu(\hat{S}_{i-1}) \leq \frac{4\varepsilon}{\log_\gamma(1/\alpha) - 2}$$

证明就是抽屉原理: 划定区间上下界 j_1, j_2 保证所有 $i \in [j_1, j_2]$ 对应的 n_i 落在 $(\alpha n, n)$ 内。总测度增量上限 4ε , 区间共 $j_2 - j_1 \geq \log_\gamma(1/\alpha) - 2$ 个分段增量, 必有一段增量不超过平均值。

定义 $k = |C_i \cap \hat{S}_{i-1}|$, C_i 中在插入前就已被判 Yes 的元素数。

引理 3.4: 满足 $k(S) \leq 9\varepsilon|C_i|$ 的序列至少占 $u^n/3$ 。

证明:

1. 假设序列逐段均匀独立从全集 U 采样。 C_i 均匀随机, $\mathbb{E}[k] = |C_i| \cdot \mu(\hat{S}_{i-1}) \leq 4\varepsilon|C_i|$ 。
2. 用切尔诺夫界: 取 $(1 + \delta) \cdot 4\varepsilon = 9\varepsilon$, 得 δ 为常数, 尾概率 $\Pr[k \geq 9\varepsilon|C_i|] \leq \exp(-|C_i|)$ 。
3. $|C_i| = n^{\Omega(1)}$, 单段超标的概率指数级小。
4. 对所有分段用联合界, 结合引理 3.2 的 $|\mathcal{S}| \geq u^n/2$, 最终满足 $k \leq 9\varepsilon|C_i|$ 的序列至少 $u^n/3$ 。

4.3.3 四段编码

对任意合法序列 S , 设计一套无损编码, 解码器仅靠编码串就能完整还原 S :

1. 下标 i : $\log \log n$ 比特。
2. C_i 以外的全部元素: 原始完整存储, $(n - c_i) \log u$ 比特 ($c_i = |C_i|$)。
3. 完整数据结构内部状态: b_i 比特 (含公共随机源 r^*)。解码器拿到前两段加上 b_i 位状态码, 可以重建 B_{r^*} 在处理 C_i 前后的内部状态, 还原 $\hat{S}_{i-1}$ 和 $\hat{S}_i$ 两个超集。

4. C_i 的相对压缩编码, 分两类:

- k 个重叠元素 (落在旧超集 $\hat{S}_{i-1}$ 内): 在 $\mu(\hat{S}_i) \cdot u$ 范围内定位, 开销 $k \cdot \log(\mu(\hat{S}_i) \cdot u) + O(1)$ 。
- $c_i - k$ 个新增元素 (落在 $\hat{S}_i \setminus \hat{S}_{i-1}$ 内): 在新增的超集区间内定位, 候选空间更小, 开销 $(c_i - k) \cdot \log(\Delta\mu \cdot u) + O(1)$ 。

总编码长度:

$$\text{TotalLen} = \log \log n + (n - c_i) \log u + b_i + (c_i - k) \log(\Delta\mu \cdot u) + k \log(\mu(\hat{S}_i)u) + O(c_i)$$

把 $\log(\mu u)$ 拆成 $\log u + \log \mu$, 提取公共 $\log u$ 项:

$$\text{TotalLen} = b_i + \log \log n + n \log u + (c_i - k) \log \Delta\mu + k \log \mu(\hat{S}_i) + O(c_i)$$

代入三个引理的约束:

- 引理 3.3: $\log \Delta\mu \leq \log \varepsilon - \log \log_\gamma(1/\alpha) + O(1)$
- $\mu(\hat{S}_i) \leq 4\varepsilon \leq \varepsilon$, 所以 $\log \mu(\hat{S}_i) \leq \log \varepsilon$
- 引理 3.4: $k \leq 9\varepsilon c_i$

合并包含 c_i 的项:

$$(c_i - k) \log \Delta\mu + k \log \mu(\hat{S}_i) \leq c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1))$$

得到简化后的上界:

$$\text{TotalLen} \leq b_i + \log \log n + n \log u + c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1))$$

4.3.4 矛盾不等式

无损编码要求能区分 $|\mathcal{S}| \geq u^n/3$ 条不同序列, 编码长度必须 $\geq \log(u^n/3) = n \log u - O(1)$ 。
令上界 $\geq$ 下界:

$$b_i + \log \log n + n \log u + c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1)) \geq n \log u - O(1)$$

消去 $n \log u$, 移项, $\log \log n$ 吸收进 c_i 的常数项, 得:

定理 3.1: 单段空间下界:

$$b_i \geq c_i \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - O(1) \right)$$

4.3.5 定理 3.1 到定理 1.1

$c_i = \gamma^i$, 等比数列求和:

$$n_i = \gamma \frac{\gamma^i - 1}{\gamma - 1} = c_i \cdot \frac{\gamma - 1/\gamma^{i-1}}{\gamma - 1}$$

$i \rightarrow \infty$ 时 $1/\gamma^{i-1} \rightarrow 0$, 得 $c_i \approx n_i \cdot (\gamma - 1)/\gamma$, 即 $c_i = \Theta(n_i)$ 。

代入反证假设 $b_i \leq \beta n_i$, 约去 n_i :

$$\beta \geq \left(1 - \frac{1}{\gamma}\right) \cdot \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - \Theta(1)\right)$$

- $\alpha = 1$ 时退化为已知规模: 令 $\gamma \rightarrow \infty$, $\beta \geq (1 - o(1)) \log(1/\varepsilon)$ ——和传统下界一致。
- $\alpha = 1/\sqrt{n}$, $\gamma = 2^{(\log n)^\eta}$ ($\eta \in (0, 1)$):

$$\log_\gamma \frac{1}{\alpha} = \log_{2^{(\log n)^\eta}} \sqrt{n} = \frac{\frac{1}{2} \log n}{(\log n)^\eta} = \frac{1}{2} (\log n)^{1-\eta} = \Theta(\log \log n)$$

回代 $S = \beta n$:

$$S \geq (1 - o(1)) n \log \frac{1}{\varepsilon} + \Omega(n \log \log n)$$

4.4 证明链路小结

1. 原始随机近似结构 $\rightarrow$ 马尔可夫不等式 (引理 3.2) $\rightarrow$ 固定 r^* 转为确定性结构
2. 对插入序列指数分段 $C_i, S_i \rightarrow$ 抽屉原理 (引理 3.3): 单段超集增量可控
3. 均匀采样 + 切尔诺夫界 (引理 3.4): 绝大多数分段重叠元素数量可控
4. 构造四段无损编码 $\rightarrow$ 信息论编码下界建立矛盾不等式
5. 化简得到单段 b_i 下界 $\rightarrow$ 结合 c_i, n_i 等比关系证明定理 3.1
6. 代入两组参数 $\rightarrow$ 分别得到已知/未知规模空间下界 (定理 1.1)

5 构造一: 几何增长数据结构

5.1 直觉

既然不知道总共要存多少元素, 那就每当前这层满了, 就开容量翻倍的新一层。每一块越来越精确, 这样总误差就可以控制在常数倍 ε 附近。

5.2 思路

多级几何扩容过滤器: 插入流按 2^i 指数分块, 第 i 块 2^i 个元素, 由专门 B_i 独立处理, 假阳性率 $\varepsilon_i = \Theta(\varepsilon/i^2)$, 总假阳性由联合界控制。每块用 Raman-Rao 紧凑字典 + 通用哈希实现。查询遍历全部 B_i , 任一返回 Yes 就判定存在。空间匹配下界, 时间 $O(\log n)$ 。

5.3 详细设计与推导

5.3.1 B_i 内部设计及已知属性

每个 B_i 使用 dictionary-based approximate membership 方法，底层用 Raman and Rao [RR03] 的动态字典：

- 选哈希函数 $h : U \rightarrow [m]$, $m \approx 2^i / \varepsilon_i$ 。
- 插入 x ：算 $h(x)$ ，把 $h(x)$ 存入 Raman-Rao 字典。
- 查询 x ：算 $h(x)$ ，查字典里有没有。
- Raman-Rao 字典：期望摊还 $O(1)$ 插入，最坏 $O(1)$ 查询，空间 $(1 + o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ bits。其空间来自信息论下界 $B = \lceil \lg \binom{m}{n} \rceil \approx n \log(m/n)$ ，代入 $m = 2^i / \varepsilon_i, n = 2^i$ 即得。

5.3.2 插入、查询与假阳性控制

插入：将要插入第 m 个元素时，通过 $2^i - 2 < m \leq 2^{i+1} - 2$ 确定该去哪个 B_i 。如果是该块第一个元素就新建 B_i ，否则直接插入。新建 B_i 的时间被摊销，期望摊还 $O(1)$ 。

查询：对当前所有已存在的 $B_1, \dots, B_k$ 逐个查询，任一 Yes 则返回 Yes。 $k \approx \log n$ ，总查询 $O(\log n)$ 。

假阳性率：由联合界，总假阳性率 $\leq \sum_{i=1}^{\infty} \varepsilon_i = \Theta(\varepsilon \cdot \pi^2/6) \approx \varepsilon$ 。

5.3.3 空间与时间复杂度

定理 4.1：对任意 $0 < \varepsilon < 1$ ，存在假阳性率 $\leq \varepsilon$ 的未知大小近似成员数据结构，满足：

1. 插入期望摊还 $O(1)$ ， n 次插入后查询 $O(\log n)$ 。
2. n 次插入后空间至多 $(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$ bits。

证明：单个 B_i 空间 $(1 + o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ 。代入 $\varepsilon_i = \Theta(\varepsilon/i^2)$ ：

$$\log(1/\varepsilon_i) = \log(1/\varepsilon) + 2 \log i + O(1)$$

求和中，每元素开销不超过最贵的那个，因此总空间：

$$\sum_i (1 + o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i) \leq (1 + o(1)) \cdot n \cdot \max_{1 \leq i \leq \lceil \log n \rceil} \{\log(1/\varepsilon_i)\}$$

$i \leq \log n$ ，取 $i = \log n$ 时 $\log(i^2) = 2 \log \log n = O(\log \log n)$ ，故：

$$\max_i \{\log(1/\varepsilon_i)\} = \log(1/\varepsilon) + O(\log \log n)$$

$$\text{总空间} = (1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$$

优点：实现简单直观。**缺点：**查询 $O(\log n)$ ，因为不知道元素属于哪个子序列。

6 构造二：常数时间查询

6.1 直觉

为解决构造 1 的查询需要对所有数据结构逐一查的问题，决定只维护一个字典，但它的“分辨率”逐步提升。同时尝试进一步消除过渡时的双倍空间。

6.2 思路

单字典 + 成对独立哈希 + 前缀扩展：任意时刻只有一个活跃字典 D_i ，元素被哈希为 (签名 h_i ，标签 g_i) 存入。当前字典满时过渡到翻倍后的字典，每次过渡时，旧元素的签名要么吸收 1 位标签位（标签位非空），要么翻倍模糊存储（标签位耗尽）。查询时直接查询当前字典即可。支持桶化优化过渡时空间和进行改造以允许删除。

6.3 详细设计与推导

6.3.1 字典设计与哈希设置

使用 pairwise independent 哈希 $h : U \rightarrow \{0, 1\}^\ell$ ， $\ell \geq \lceil \log(1/\varepsilon) \rceil + \log u + 2$ 。

同样将插入序列按 2^i 分块。对每个阶段 i 的元素 x ，在 D_i 中存储 $(h_i(x), g_i(x))$ ：

- $h_i(x)$ ： $h(x)$ 的最左 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$ 位，作为字典键。
- $g_i(x)$ ： $h(x)$ 的接下来 $r = \lceil \log \log u \rceil$ 位，不足时以 $\perp$ 填充。

6.3.2 插入、查询与无假阴性

Mode 1 (普通插入)：当前元素不是子序列的第一个。直接把 $(h_i(x), g_i(x))$ 以 $h_i(x)$ 为键插入当前 D_i 。 $O(1)$ 。

Mode 2 (阶段过渡)：当前元素是第 i 个子序列的第一个。做字典切换：

1. 初始化 D_i ：容量 2^{i+2} 项，每项 $\ell_i + r$ 位。
2. 枚举 D_{i-1} 中所有记录 $(y, \alpha_1 \cdots \alpha_r)$ 迁移：
 - 若 $\alpha_1 \neq \perp$ ：插入 $(y\alpha_1, \alpha_2 \cdots \alpha_r \perp)$ ，扩展一位签名。
 - 若 $\alpha_1 = \perp$ ：分支，插入 $(y0, \alpha)$ 和 $(y1, \alpha)$ 。
3. 释放 D_{i-1} ，然后按 Mode 1 插入当前元素。

查询：当前阶段 i ，用 $h_i(x)$ 查 D_i ，命中则 Yes，否则 No。查询最坏 $O(1)$ 。

无假阴性：若 $\alpha_1 = \perp$ 则同时保留 $y0$ 和 $y1$ ，覆盖了 $h_i(x)$ 所有可能前缀；若 $\alpha_1 \neq \perp$ 则延长 $y\alpha_1$ ，始终跟随实际哈希值。因此 x 在任意后续阶段都至少有 $(h_i(x), \cdot)$ 在 D_i 中，查询一定命中。

6.3.3 失败处理与桶化

基本构造中从 D_i 过渡到 D_{i+1} 时两者需同时存在，空间暂时翻倍。用桶化处理：

- 哈希把元素分摊到 $u^{\delta/2}$ 个桶。
- 各桶数据结构按字交错存储 (interleaved)，保证过渡操作至多在一个桶中发生。
- 于是额外空间只与单个桶大小成比例。

6.3.4 空间复杂度

定理 5.1: 对任意 $0 < \varepsilon < 1$ 、整数 u 及常数 $c > 1$ ，存在假阳性率 $\varepsilon + u^{-c}$ 的未知大小近似成员数据结构。对任意 $0 < \delta < 1$ 和 $n > u^\delta$ 次插入，满足：

1. 假阳性率 $\leq \varepsilon + u^{-c}$ 。
2. 空间至多 $(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$ bits。
3. 插入期望摊还 $O(1)$ ，查询最坏情况 $O(1)$ 。

空间的证明： 以下对单个桶分析。桶内有 n 个元素 ($n > u^\delta$)，设当前处于第 i 个子序列期间，即 $2^{i-1} \leq n \leq 2^i - 1$ ，当前字典为 D_i 。

将 D_i 中的条目按其来源子序列分为两类：

早期子序列 $(S_1, \dots, S_{i-r})$ ：标签 $g_i(x)$ 中 r 位已耗尽 ($\alpha_1 = \perp$)，每次过渡翻倍。每个 S_j 元素贡献 2^{i-j-r} 个条目。总空间：

$$\sum_{j=1}^{i-r} |S_j| \cdot 2^{i-j-r} = \sum_{j=1}^{i-r} 2^{i-r-1}$$

近期子序列 $(S_{i-r+1}, \dots, S_i)$ ：标签尚有剩余 ($\alpha_1 \neq \perp$)，每次过渡仅扩展 1 位不翻倍，每元素恰好贡献 1 条目。总空间：

$$\sum_{j=i-r+1}^i 2^j$$

总条目数：

$$|D_i| = \sum_{j=1}^{i-r} 2^{i-r-1} + \sum_{j=i-r+1}^i 2^j \leq (i-r) \cdot 2^{i-r-1} + 2^{i+1} \leq 2^{i+2}$$

因此每个桶分配的容量 2^{i+2} 是足够的。

D_i 使用 RR03 紧凑动态字典，存储了 n 个元素。每个元素的形式为 (key, sat)：键是 ℓ_i 位签名，来自全集大小 2^{ℓ_i} ；标签 r 位。RR03 的空间保证为 $(1 + o(1)) \cdot n \cdot (\log(u/n) + r)$ 。代入 $u = 2^{\ell_i}$ ，并以 2^i 近似 n ：

$$|D_i| \leq (1 + o(1)) \cdot n \cdot (\log(2^{\ell_i}/2^i) + r) = (1 + o(1)) \cdot n \cdot (\ell_i - i + r)$$

代入 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$, 即 $\ell_i - i = \log(1/\varepsilon) + O(1)$; 代入 $r = \lceil \log \log u \rceil$, 即由 $n > u^\delta$ 保证的 $\log \log u = \log \log n + O(1)$ 。因此:

$$|D_i| \leq (1 + o(1)) \cdot n \cdot (\log(1/\varepsilon) + r + O(1)) = (1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$$

假阳性率的证明: D_i 中最多 2^{i+2} 个键。由 pairwise independent 哈希, $h_i(x)$ 与任一已有键碰撞的概率 $\leq 2^{i+2} \cdot 2^{-\ell_i}$ 。代入 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$:

$$2^{i+2} \cdot 2^{-\ell_i} \leq 2^{i+2} \cdot \frac{\varepsilon}{2^{i+2}} = \varepsilon$$

底层动态字典 D_i 有一定失败概率, 通过将前 u^δ 个元素合并到一个字典, 联合界保证总失败概率 $\leq u^{-c}$ 。因此假阳性率 $\leq \varepsilon + u^{-c}$ 。

6.4 扩展: 去摊销化与删除支持

6.4.1 去摊销化

不等到子序列第一个元素时一次性初始化 D_i , 而是把初始化工作均摊到该子序列的 2^i 次插入中。每次插入额外投入常数步工作。代价是不能用桶化, 空间从 $(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$ 升到 $O(n \log(1/\varepsilon) + n \log \log n)$ 。

6.4.2 支持删除

近似成员结构无法检测是否删的是假阳性, 需要引入辅助性的精确字典判断。

- 主字典: 同原结构, 每个条目多一个删除指示位。
- 辅助字典: 存”发现是假阳性”的元素的精确值。

插入时先检查是不是假阳性, 是则放辅助字典, 否则正常插入主字典。删除时先查辅助字典, 有则清除该条目, 没有则查主字典, 标记最小字典序条目为已删除。

7 相关研究

7.1 论文提到的五类相关研究

7.1.1 Bloom filters (布隆过滤器)

- 布隆过滤器是一种很经典的数据结构, 天然支持动态插入。其空间开销是信息论下界 $n \log(1/\varepsilon)$ 的 $\log(e) \approx 1.44$ 倍。
- 标准的 Bloom 过滤器不支持从集合 S 中删除元素, 因为将任意位重置为 0 可能会导致假阴性。

- 为了支持删除查询，可以使用计数布隆过滤器。但这会使空间使用增加 $\Omega(\log \log n)$ 倍。这种删除支持是有条件的：只有在不删除假阳性元素时，数据结构才能正确工作，但根据定义，无法完全避免对假阳性元素的删除。
- Cohen 和 Matias 提出了一种方法，将空间开销降低至 $O(n)$ 位。

7.1.2 基于字典的近似成员查询

- 早在 1978 年，Carter 等人就提出了一种能达到类似结果的技术。他们观察到，维护一个多重集 $h(S)$ （其中 $h: [u] \rightarrow [n/\varepsilon]$ 是一个通用哈希函数），如果集合 $h(S)$ 的存储空间接近信息论下界 $\log \binom{n/\varepsilon+n}{n}$ 位，那么整体解决方案的空间消耗仅为 $n \log(1/\varepsilon) + O(n)$ 位。
- 如果不需要支持删除操作，只需存储不同的哈希值集合 $h(S)$ 即可。这种动态集合可以被简洁地存储，且所有操作以高概率在 $O(1)$ 时间内完成。如果需要支持动态多重集（即支持删除），可以通过转化为标准成员查询问题来实现，代价是更新操作的时间增大。

7.1.3 在线/离线空间需求的差异

- 在静态情况下，已经证明能够逼近 $n \log(1/\varepsilon)$ 的空间下界。例如，Dietzfelbinger 和 Pagh 证明了在查询时间为 $\omega(\log(1/\varepsilon))$ 且 ε 为 2 的整数幂时，空间开销可以控制在 $o(n)$ 项以内；Porat 在常数查询时间的情况下实现了相同的结果；随后 Bellazougui 和 Venturini 进一步消除了对 ε 的限制，同时维持了常数查询时间。
- 然而，Lovett 和 Porat 证明了这些静态情况下的结果无法直接扩展到允许动态场景：在动态更新下，需要 $\Omega(n/\log(1/\varepsilon))$ 位的额外空间开销。即使在整个插入序列结束前不进行任何查询，该下界依然成立。这意味着，对于以数据流形式给出的键集，仅使用接近静态大小下界的空间来构建近似成员数据结构是不够的。

7.1.4 动态空间使用

- 当要求空间必须依赖于集合的当前大小时，上界要求更加严格。
- 文献中现有的相关技术通常会导致 $\Omega(\log n)$ 或 $\Omega(\log(1/\varepsilon))$ 的查询时间，以及 $\Omega(n \log n)$ 位的高空间开销。
- 这些数据结构通常采用维护多个近似成员数据结构并逐一查询的策略。如果采用几何级数递增的容量，就需要 $\Omega(\log n)$ 个这样的数据结构。每一个结构对应的假阳性率 $\varepsilon_1, \varepsilon_2, \dots$ 之和必须收敛于目标 ε 。例如，在已有的 Scalable Bloom Filter 中让 ε_i 随 i 呈几何级数递减，会导致 $\varepsilon_i = n^{-\Omega(i)}$ ，从而产生 $\Omega(n \log n)$ 的空间使用量——渐近上远非最优。

7.1.5 动态完美哈希与检索

- 在静态情况下，一种实现近似成员查询的方法是存储一个完美哈希函数，将键单射地映射到 $\{1, \dots, n\}$ ，然后在数组中根据该完美哈希函数为每个键存储 $\log(1/\varepsilon)$ 位的签名。

- 更一般地，动态检索数据结构（如 Bloomier filters）也可以用来构建动态近似成员数据结构。研究表明，在线设置下，这两个问题都需要 $\Theta(n \log \log n)$ 的空间。然而，这些上界具有固定的空间使用量（在常数因子内），因此无法实现本文所获得的结果。

7.2 发表后的新研究

下列三项后续工作均引用了本文，且均在“集合规模事先未知”这一设定下做了进一步推进。

Aleph Filter: To Infinity in Constant Time (Dayan, Bercea, Pagh, PVLDB 2024)

Aleph Filter 是一个可无限扩展的近似成员过滤器，无论数据增长多少，插入、查询和删除均保持常数时间。若提前知道数据规模上限，其内存与假阳性率可匹敌静态过滤器。该工作在 §7 Related Work 中明确引用了本文的下界，并声明其 Widening 和 Predictive 两种工作模式均达到该下界。合著者 Rasmus Pagh 同时是本文的第一作者，Aleph Filter 视为对本文理论结果的工程实现。

Resizable Retrieval (Kuszmaul et al., 2026)

该工作回答了 Demaine et al. 2006 提出的开放问题：能否将动态检索结构的空間上界从预设容量 N 替换为当前实际大小 n 。论文给出了肯定答案，并证明了匹配的空间-时间 tradeoff 下界。其关键技术之一——“碰撞风险分摊”——直接延续了本文 [PSW13] 中的技术路线（见论文 §1）。作为推论，该工作还给出了动态过滤器的一个新构造。

Dynamic “Succincter” (Li, Liang, Yu, Zhou, FOCS 2023)

该工作将 Pătraşcu 2008 的静态 Succincter 框架动态化：提出的动态 aB-tree (daB-tree) 仅需 3 bit 冗余，作为应用构造了更新/查询时间均最优的动态完全可索引字典。该工作在参考文献中将本文列为紧凑字典与过滤器领域的背景工作。它与本文属于正交关系——本文解决“未知 n ”的框架问题，Dynamic Succincter 优化的是底层字典的冗余常数。两者可叠加，但互不替代。

8 总结与讨论

8.1 主要贡献回顾

本文在动态近似成员查询的“未知 n ”设定上完成了三项工作。第一，通过压缩论证证明了空间下界 $(1 - o(1))n \log(1/\varepsilon) + \Omega(n \log \log n)$ ，表明每元素的平均存储开销不可能独立于 n ——这是此前未被严格证明的结论。第二，给出构造一（几何增长过滤器），以多项式衰减的假阳性率序列 $\varepsilon_i = \Theta(\varepsilon/i^2)$ 替代旧方案的指数衰减，将附加空间从 $\Omega(n \log n)$ 压至 $O(n \log \log n)$ ，代价是查询时间 $O(\log n)$ 。第三，提出构造二（单字典前缀扩展），用成对独立哈希和 $r = \lceil \log \log u \rceil$

位辅助标签控制签名翻倍, 实现查询 $O(1)$ 最坏情况、插入摊还 $O(1)$ 、空间同阶的结果; 去摊销化将其升级为插入最坏 $O(1)$ (空间略增至 $O(n \log(1/\varepsilon) + n \log \log n)$)。

上下界在渐近阶上完全一致(下界 Ω 、上界 O , 差仅低阶项), 说明本文下界是紧的, $\Omega(n \log \log n)$ 是不可消除的固有代价。

8.2 $\log \log n$ 的意义

从理论角度看, $\Omega(n \log \log n)$ 项确立了”已知 n ”与”未知 n ”两种模型之间确实存在超线性空间差距。这个附加开销并不大: $n = 2^{64}$ 时 $\log \log n$ 仅为 6, 而构造一的旧方案付出的是 $\log n$ 级别 (≈ 64), 改进幅度显著。从工程角度看, 本文首次给出了自适应扩容结构的固有存储下界, 后续工作 (如 Aleph Filter 和 Resizable Retrieval) 已在此理论上给出了实用性更强的构造。

8.3 论文指出的未来方向

- 理论: 收紧 $\Omega(n \log \log n)$ 附加项的常数因子。
- 实践: 构造 1 查询 $O(\log n)$ 较慢, 构造 2 隐藏常数较大。期待新设计同时满足最优空间和常数时间的实用数据结构, 匹配本文所证明的空间下界。